Конспект. Стандарты форматирования сообщений

1. Назначение форматов сообщений

Формат сообщения определяет структуру данных, правила кодирования и декодирования, а также смысл полей. Транспортные протоколы (TCP, UDP, QUIC) доставляют байты, но сами по себе не объясняют их значение. Неудачный формат увеличивает объём данных и задержки, усложняет изменение структуры и снижает совместимость систем.

Основные понятия

Сериализация — преобразование структуры данных в последовательность байтов. Десериализация — обратное преобразование. Схема описывает поля, типы и ограничения. Схемная эволюция позволяет изменять схему с сохранением совместимости. Обратная совместимость означает, что новая версия читает старые данные; прямая — что старая версия читает новые данные. Детерминизм — получение одинаковой последовательности байтов для одинаковых данных.

2. Классификация

Текстовые форматы (JSON, XML, YAML, TOML) удобны для чтения, отладки и ручного редактирования, но обычно более объёмны и медленнее бинарных. Бинарные форматы (Protobuf, Avro, Thrift, MessagePack, CBOR, FlatBuffers, Cap’n Proto) ориентированы на компактность и машинную обработку. Схемные форматы требуют описания структуры; бессхемные позволяют передавать её непосредственно в сообщении.

3. Текстовые форматы

JSON. Представляет объекты как пары «ключ — значение» и массивы. Плюсы: простота, широкая поддержка, читаемость, удобство для REST API и веб-приложений. Минусы: повторение ключей, отсутствие комментариев и строгой схемной эволюции, более медленная обработка по сравнению с некоторыми бинарными форматами.

XML. Использует теги и древовидную структуру; поддерживает схемы XSD и пространства имён. Применяется в SOAP, документообороте и корпоративных/устаревших системах. Недостатки — многословность и более сложный разбор.

YAML. Читаемый текстовый формат с поддержкой комментариев, часто применяется для конфигураций Docker Compose, Kubernetes и CI/CD. Чувствителен к отступам и может медленно обрабатываться.

4. Бинарные форматы

Protocol Buffers (Protobuf) — бинарный формат Google с явной схемой .proto. Быстрый и компактный, поддерживает типизацию, генерацию кода и добавление полей с сохранением совместимости. Минусы: нужна схема, данные неудобны для чтения человеком.

Apache Avro использует схемы в JSON, поддерживает эволюцию и широко применяется в Big Data, в том числе с Kafka и Spark. MessagePack похож на модель JSON, но кодирует данные компактнее; отдельная схема не обязательна. CBOR стандартизован IETF, компактен и подходит для IoT и ограниченных устройств. Apache Thrift сочетает сериализацию и RPC, использует IDL и поддерживает много языков.

5. Как выбирать формат

• Производительность и задержки: при высоких нагрузках рассматривать бинарные форматы; для простых задач часто достаточно JSON.
• Компактность: CBOR, Protobuf или MessagePack для ограниченной пропускной способности.
• Читаемость: JSON для API, YAML для конфигураций.
• Эволюция схем: Avro или Protobuf.
• Совместимость: XML для систем, где он уже требуется; JSON для веб-API; Protobuf для gRPC.
• Экосистема: Avro часто используют с Kafka и Hadoop, Protobuf — с gRPC.
• Детерминизм: важен там, где требуется воспроизводимость байтов.

6. Стандарты и практические рекомендации

Форматы и отраслевые протоколы описываются стандартами и спецификациями. В лекции приведены JSON (RFC 8259), CBOR (RFC 8949), а также FIX для финансов, HL7 для здравоохранения и ARINC для авиации. Типичные ошибки — выбирать формат по популярности, использовать чрезмерно сложный формат для простой задачи, не поддерживать схему Protobuf и игнорировать совместимость при изменении данных.

Рекомендации из лекции: JSON для REST API; Protobuf для gRPC; Avro для Kafka; CBOR или MessagePack для IoT; YAML для конфигураций; XML для существующих legacy-систем. Schema Registry хранит версии схем, проверяет совместимость и предоставляет схемы производителям и потребителям.